JOBSHEET XI

Mata Kuliah: Praktikum Algoritma Dan Struktur Data

LINKED LIST

Dosen Pengampu:

Mungki Astiningrum, S.T., M.Kom.

Disusun oleh:

Ahmad Rafid Riqkullah

254107020078

T1 – 1G

D-IV Teknik Informatika

POLITEKNIK NEGERI MALANG

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141

TAHUN 2025-2026

2. Praktikum

2.1 Pembuatan Single Linked List

- **Mahasiswa03.java**

```
package Jobsheet11.Pertemuan12;

public class Mahasiswa03 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    public Mahasiswa03() {
    }

    public Mahasiswa03(String nim, String nama, String kelas, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    public void tampilInformasi() {
        System.out.println(nama + "\t" + nim + "\t" + kelas + "\t" + ipk);
    }
}
```

- **NodeMahasiswa03.java**

```
package Jobsheet11.Pertemuan12;

public class NodeMahasiswa03 {
    Mahasiswa03 data;
    NodeMahasiswa03 next;

    public NodeMahasiswa03(Mahasiswa03 data, NodeMahasiswa03 next) {
        this.data = data;
        this.next = next;
    }
}
```

- **SingleLinkedList03.java**

```
package Jobsheet11.Pertemuan12;

public class SingleLinkedList03 {
    NodeMahasiswa03 head;
    NodeMahasiswa03 tail;

    boolean isEmpty() {
        return (head == null);
    }
```

```
public void print() {
    if (!isEmpty()) {
        NodeMahasiswa03 tmp = head;
        System.out.println("Isi Linked List:");
        while (tmp != null) {
            tmp.data.tampilInformasi();
            tmp = tmp.next;
        }
        System.out.println("");
    } else {
        System.out.println("Linked list kosong");
    }
}

public void addFirst(Mahasiswa03 input) {
    NodeMahasiswa03 ndInput = new NodeMahasiswa03(input, null);
    if (isEmpty()) {
        head = ndInput;
        tail = ndInput;
    } else {
        ndInput.next = head;
        head = ndInput;
    }
}

public void addLast(Mahasiswa03 input) {
    NodeMahasiswa03 ndInput = new NodeMahasiswa03(input, null);
    if (isEmpty()) {
        head = ndInput;
        tail = ndInput;
    } else {
        tail.next = ndInput;
        tail = ndInput;
    }
}

public void insertAfter(String key, Mahasiswa03 input) {
    NodeMahasiswa03 ndInput = new NodeMahasiswa03(input, null);
    NodeMahasiswa03 temp = head;
    do {
        if (temp.data.nama.equalsIgnoreCase(key)) {
            ndInput.next = temp.next;
            temp.next = ndInput;
            if (ndInput.next == null) {
                tail = ndInput;
            }
            break;
        }
        temp = temp.next;
    } while (temp != null);
```

```
    }

    public void insertAt(int index, Mahasiswa03 input) {
        if (index < 0) {
            System.out.println("indeks salah");
        } else if (index == 0) {
            addFirst(input);
        } else {
            NodeMahasiswa03 temp = head;
            for (int i = 0; i < index - 1; i++) {
                temp = temp.next;
            }
            temp.next = new NodeMahasiswa03(input, temp.next);
            if (temp.next.next == null) {
                tail = temp.next;
            }
        }
    }
}
```

- **SLLMain03.java**

```
package Jobsheet11.Pertemuan12;

public class SLLMain03 {
    public static void main(String[] args) {
        SingleLinkedList03 sll = new SingleLinkedList03();

        Mahasiswa03 mhs1 = new Mahasiswa03("24212200", "Alvaro", "1A", 4.0);
        Mahasiswa03 mhs2 = new Mahasiswa03("23212201", "Bimon", "2B", 3.8);
        Mahasiswa03 mhs3 = new Mahasiswa03("22212202", "Cintia", "3C", 3.5);
        Mahasiswa03 mhs4 = new Mahasiswa03("21212203", "Dirga", "4D", 3.6);

        sll.print();
        sll.addFirst(mhs4);
        sll.print();
        sll.addLast(mhs1);
        sll.print();
        sll.insertAfter("Dirga", mhs3);
        sll.insertAt(2, mhs2);
        sll.print();
    }
}
```

2.1.1 Verifikasi Hasil Percobaan

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>  c:; cd 'c:\Users\IdeaPad\OneDrive\Dokumen
tur Data\Praktik Algo&StrukturData'; & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Ide
\workspaceStorage\1252898021389025de57e0e2ce7124bb\redhat.java\jdt_ws\Praktik Algo&StrukturData_7951cb7c\bin' 'Jobsheet11.Pertemuan12.SLLMain03'
Linked list kosong
Isi Linked List:
Dirga   21212203       4D      3.6

Isi Linked List:
Dirga   21212203       4D      3.6
Alvaro  24212200       1A      4.0

Isi Linked List:
Dirga   21212203       4D      3.6
Cintia  22212202       3C      3.5
Bimon   23212201       2B      3.8
Alvaro  24212200       1A      4.0

PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>
```

2.1.2 Pertanyaan

1. Mengapa hasil compile kode program di baris pertama menghasilkan “Linked List Kosong”?
2. Jelaskan kegunaan variable temp secara umum pada setiap method!
3. Lakukan modifikasi agar data dapat ditambahkan dari keyboard!

Jawab

1. Hasilnya "Linked List Kosong" karena pemanggilan method sll.print() yang pertama kali dilakukan sebelum ada data mahasiswa yang dimasukkan (di-add) ke dalam struktur linked list. Karena belum ada data yang ditambahkan, pointer head masih bernilai null. Akibatnya, saat fungsi print() dicek, kondisi isEmpty() bernilai true, sehingga program langsung mengeksekusi blok else yang berisi perintah untuk mencetak teks "Linked list kosong".
2. Secara umum, variabel temp (singkatan dari temporary atau sementara) berfungsi sebagai "penunjuk jalan" untuk menyusuri (traverse) data dari awal sampai akhir linked list. Berbeda dengan Array yang datanya bisa langsung diambil pakai indeks (misal: data[2]), pada Linked List harus mengecek datanya satu per satu dari paling depan (head). Temp ini digunakan untuk melompat dari satu node ke node berikutnya (menggunakan perintah temp = temp.next) sampai menemukan posisi kosong atau data yang dicari, tanpa merusak atau menggeser posisi asli dari head.
3. Ubah pada class SSLMain03.java menjadi:

```
package Jobsheet11.Pertemuan12;

import java.util.Scanner;

public class SLLMain03 {

    public static void main(String[] args) {

        SingleLinkedList03 sll = new SingleLinkedList03();
```

```
        Scanner sc = new Scanner(System.in);
        char pilih;

        do{
        System.out.println("=== Tambah Data Mahasiswa ===");
        System.out.print("Masukkan NIM   : ");
        String nim = sc.nextLine();

        System.out.print("Masukkan Nama  : ");
        String nama = sc.nextLine();

        System.out.print("Masukkan Kelas : ");
        String kelas = sc.nextLine();

        System.out.print("Masukkan IPK   : ");
        double ipk = sc.nextDouble();

        // Buat object mahasiswa baru dari data yang diinputkan
        Mahasiswa03 mhsBaru = new Mahasiswa03(nim, nama, kelas, ipk);
        // Tambahkan ke dalam Linked List (misal ditaruh di paling belakang)
        sll.addLast(mhsBaru);

        System.out.print("Apakah ingin menambahkan data lagi? (y/n): ");
        pilih = sc.next().charAt(0);
        sc.nextLine();
        } while (pilih == 'y' || pilih == 'Y');

        System.out.println("\nData setelah ditambahkan:");
        sll.print();
        sc.close();
    }
}
```

2.2 Modifikasi Elemen pada Single Linked List

2.2.1 Langkah-langkah Percobaan

- **Mahasiswa03.java**

```
package Jobsheet11.Pertemuan12;

public class Mahasiswa03 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    public Mahasiswa03() {
    }

    public Mahasiswa03(String nim, String nama, String kelas, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    public void tampilInformasi() {
        System.out.println(nama + "\t" + nim + "\t" + kelas + "\t" + ipk);
    }
}
```

- **NodeMahasiswa03.java**

```
package Jobsheet11.Pertemuan12;

public class NodeMahasiswa03 {
    Mahasiswa03 data;
    NodeMahasiswa03 next;

    public NodeMahasiswa03(Mahasiswa03 data, NodeMahasiswa03 next) {
        this.data = data;
        this.next = next;
    }
}
```

- **SingleLinkedList03.java**

```
package Jobsheet11.Pertemuan12;

public class SingleLinkedList03 {
    NodeMahasiswa03 head;
    NodeMahasiswa03 tail;

    boolean isEmpty() {
        return (head == null);
    }
```

```
public void print() {
    if (!isEmpty()) {
        NodeMahasiswa03 tmp = head;
        System.out.println("Isi Linked List:");
        while (tmp != null) {
            tmp.data.tampilInformasi();
            tmp = tmp.next;
        }
        System.out.println("");
    } else {
        System.out.println("Linked list kosong");
    }
}

public void addFirst(Mahasiswa03 input) {
    NodeMahasiswa03 ndInput = new NodeMahasiswa03(input, null);
    if (isEmpty()) {
        head = ndInput;
        tail = ndInput;
    } else {
        ndInput.next = head;
        head = ndInput;
    }
}

public void addLast(Mahasiswa03 input) {
    NodeMahasiswa03 ndInput = new NodeMahasiswa03(input, null);
    if (isEmpty()) {
        head = ndInput;
        tail = ndInput;
    } else {
        tail.next = ndInput;
        tail = ndInput;
    }
}

public void insertAfter(String key, Mahasiswa03 input) {
    NodeMahasiswa03 ndInput = new NodeMahasiswa03(input, null);
    NodeMahasiswa03 temp = head;
    do {
        if (temp.data.nama.equalsIgnoreCase(key)) {
            ndInput.next = temp.next;
            temp.next = ndInput;
            if (ndInput.next == null) {
                tail = ndInput;
            }
            break;
        }
        temp = temp.next;
    } while (temp != null);
```

```
}

public void insertAt(int index, Mahasiswa03 input) {
    if (index < 0) {
        System.out.println("indeks salah");
    } else if (index == 0) {
        addFirst(input);
    } else {
        NodeMahasiswa03 temp = head;
        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }
        temp.next = new NodeMahasiswa03(input, temp.next);
        if (temp.next.next == null) {
            tail = temp.next;
        }
    }
}

public void getData(int index) {
    NodeMahasiswa03 tmp = head;
    for (int i = 0; i < index; i++) {
        tmp = tmp.next;
    }
    tmp.data.tampilInformasi();
}

public int indexOf(String key) {
    NodeMahasiswa03 tmp = head;
    int index = 0;
    while (tmp != null && !tmp.data.nama.equalsIgnoreCase(key)) {
        tmp = tmp.next;
        index++;
    }
    if (tmp == null) {
        return -1;
    } else {
        return index;
    }
}

public void removeFirst() {
    if (isEmpty()) {
        System.out.println("Linked List masih Kosong, tidak dapat dihapus!");
    } else if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
    }
}
```

```
public void removeLast() {
    if (isEmpty()) {
        System.out.println("Linked List masih Kosong, tidak dapat dihapus!");
    } else if (head == tail) {
        head = tail = null;
    } else {
        NodeMahasiswa03 temp = head;
        while (temp.next != tail) {
            temp = temp.next;
        }
        temp.next = null;
        tail = temp;
    }
}

public void remove(String key) {
    if (isEmpty()) {
        System.out.println("Linked List masih Kosong, tidak dapat dihapus!");
    } else {
        NodeMahasiswa03 temp = head;
        while (temp != null) {
            if ((temp.data.nama.equalsIgnoreCase(key)) && (temp == head)) {
                this.removeFirst();
                break;
            } else if (temp.next != null && temp.next.data.nama.equalsIgnoreCase(key)) {
                temp.next = temp.next.next;
                if (temp.next == null) {
                    tail = temp;
                }
                break;
            }
            temp = temp.next;
        }
    }
}

public void removeAt(int index) {
    if (index == 0) {
        removeFirst();
    } else {
        NodeMahasiswa03 temp = head;
        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }
        temp.next = temp.next.next;
        if (temp.next == null) {
            tail = temp;
        }
    }
```

```
    }
}
```

- **SLLMain03.java**

```
package Jobsheet11.Pertemuan12;

import java.util.Scanner;

public class SLLMain03 {
    public static void main(String[] args) {
        SingleLinkedList03 sll = new SingleLinkedList03();

        // Pembuatan 4 object mahasiswa
        Mahasiswa03 mhs1 = new Mahasiswa03("24212200", "Alvaro", "1A", 4.0);
        Mahasiswa03 mhs2 = new Mahasiswa03("23212201", "Bimon", "2B", 3.8);
        Mahasiswa03 mhs3 = new Mahasiswa03("22212202", "Cintia", "3C", 3.5);
        Mahasiswa03 mhs4 = new Mahasiswa03("21212203", "Dirga", "4D", 3.6);

        // --- KODE PRAKTIKUM 2.1 ---
        sll.print();
        sll.addFirst(mhs4);
        sll.print();
        sll.addLast(mhs1);
        sll.print();
        sll.insertAfter("Dirga", mhs3);
        sll.insertAt(2, mhs2);
        sll.print();

        // --- KODE PRAKTIKUM 2.2 ---
        System.out.println("data index 1: ");
        sll.getData(1);

        System.out.println("data mahasiswa an Bimon berada pada index: " +
sll.indexOf("Bimon"));
        System.out.println();

        sll.removeFirst();
        sll.removeLast();
        sll.print();

        sll.removeAt(0);
        sll.print();
    }
}
```

2.2.2 Verifikasi Hasil Percobaan

```
PROBLEMS 1   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData> ^C
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>  c:; cd 'c:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struk
tur Data\Praktik Algo&StrukturData'; & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\IdeaPad\AppData\Roaming\Code\User
\workspaceStorage\125289802138902 5de57e0e2ce7124bb\redhat.java\jdt_ws\Praktik Algo&StrukturData_7951cb7c\bin' 'Jobsheet11.Pertemuan12.SLLMain03'
Linked list kosong
Isi Linked List:
Dirga   21212203        4D      3.6

Isi Linked List:
Dirga   21212203        4D      3.6
Alvaro  24212200        1A      4.0

Isi Linked List:
Dirga   21212203        4D      3.6
Cintia  22212202        3C      3.5
Bimon   23212201        2B      3.8
Alvaro  24212200        1A      4.0

data index 1:
Cintia  22212202        3C      3.5
data mahasiswa an Bimon berada pada index: 2

Isi Linked List:
Cintia  22212202        3C      3.5
Bimon   23212201        2B      3.8

Isi Linked List:
Bimon   23212201        2B      3.8

PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>
```

2.2.3 Pertanyaan

1. Mengapa digunakan keyword break pada fungsi remove? Jelaskan!
2. Jelaskan kegunaan kode dibawah pada method remove

```
temp.next = temp.next.next;
if (temp.next == null) {
    tail = temp;
}
```

Jawab

1. Keyword break di situ berfungsi untuk langsung menghentikan proses perulangan (while) seketika setelah data yang dicari berhasil ditemukan dan dihapus.

2. **temp.next = temp.next.next;** Baris ini adalah eksekusi penghapusan node. Pointer next dari node saat ini (temp) diubah agar langsung menunjuk ke node *setelah* node target penghapusan (temp.next.next). Dengan memutus koneksi referensi ini, node yang berada di tengah-tengah tidak lagi terhubung di dalam linked list dan nantinya akan dibersihkan.

   if (temp.next == null) { tail = temp; } Blok percabangan ini berfungsi untuk memperbarui pointer tail jika penghapusan terjadi di ujung list. Program mengecek apakah setelah memutus koneksi tadi, pointer temp sekarang menunjuk ke null. Jika kondisinya true (bernilai null), berarti node yang dihapus sebelumnya adalah node terakhir. Oleh karena itu, pointer tail harus diperbarui untuk menunjuk ke temp agar status akhir struktur linked list tetap valid.

3. Tugas

Buatlah implementasi program antrian layanan unit kemahasiswaan sesuai dengan berikut ini :

a. Implementasi antrian menggunakan Queue berbasis Linked List!
b. Program merupakan proyek baru bukan modifikasi dari percobaan
c. Ketika seorang mahasiswa akan mengantri, maka dia harus mendaftarkan datanya
d. Cek antrian kosong, Cek antrian penuh, Mengosongkan antrian.
e. Menambahkan antrian
f. Memanggil antrian
g. Menampilkan antrian terdepan dan antrian paling akhir
h. Menampilkan jumlah mahasiswa yang masih mengantre.

Jawab

- **MahasiswaLayanan03.java**

```
package Jobsheet11.Tugas;

public class MahasiswaLayanan03 {
    String nim;
    String nama;

    public MahasiswaLayanan03(String nim, String nama) {
        this.nim = nim;
        this.nama = nama;
    }

    public void tampilData() {
        System.out.println("NIM: " + nim + " | Nama: " + nama);
    }
}
```

- **NodeAntrian03.java**

```
package Jobsheet11.Tugas;

public class NodeAntrian03 {
    MahasiswaLayanan03 data;
    NodeAntrian03 next;

    public NodeAntrian03(MahasiswaLayanan03 data, NodeAntrian03 next) {
        this.data = data;
        this.next = next;
    }
}
```

- **QueueLayanan03.java**

```
package Jobsheet11.Tugas;
```

```
public class QueueLayanan03 {
    NodeAntrian03 front, rear;
    int size;
    int kapasitasMaksimal;

    public QueueLayanan03(int kapasitasMaksimal) {
        this.front = null;
        this.rear = null;
        this.size = 0;
        this.kapasitasMaksimal = kapasitasMaksimal;
    }

    // d. Cek antrian kosong
    public boolean isEmpty() {
        return size == 0;
    }

    // d. Cek antrian penuh
    public boolean isFull() {
        return size == kapasitasMaksimal;
    }

    // d. Mengosongkan antrian
    public void clear() {
        front = rear = null;
        size = 0;
        System.out.println("Semua antrian telah dikosongkan.");
    }

    // e. Menambahkan antrian (Enqueue / Insert di belakang)
    public void enqueue(MahasiswaLayanan03 mhs) {
        if (isFull()) {
            System.out.println("Mohon maaf, kapasitas antrian sudah penuh!");
        } else {
            NodeAntrian03 newNode = new NodeAntrian03(mhs, null);
            if (isEmpty()) {
                front = rear = newNode;
            } else {
                rear.next = newNode;
                rear = newNode;
            }
            size++;
            System.out.println("Berhasil menambahkan " + mhs.nama + " ke dalam antrian.");
        }
    }

    // f. Memanggil antrian (Dequeue / Remove dari depan)
    public void dequeue() {
        if (isEmpty()) {
            System.out.println("Tidak ada antrian yang bisa dipanggil (antrian kosong).");
```

```
        } else {
            MahasiswaLayanan03 dipanggil = front.data;
            front = front.next;
            if (front == null) {
                rear = null;
            }
            size--;
            System.out.println("Memanggil antrian atas nama:");
            dipanggil.tampilData();
            System.out.println("Silakan menuju loket layanan.");
        }
    }

    // g. Menampilkan antrian terdepan dan antrian paling akhir
    public void printFrontAndRear() {
        if (isEmpty()) {
            System.out.println("Antrian kosong.");
        } else {
            System.out.println("--- Posisi Antrian Saat Ini ---");
            System.out.print("Antrian Terdepan : ");
            front.data.tampilData();
            System.out.print("Antrian Terakhir : ");
            rear.data.tampilData();
        }
    }

    // h. Menampilkan jumlah mahasiswa yang masih mengantre
    public void printQueueCount() {
        System.out.println("Jumlah mahasiswa yang masih mengantre: " + size + " orang.");
    }
}
```

- **MainLayanan03.java**

```
package Jobsheet11.Tugas;

import java.util.Scanner;

public class MainLayanan03 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        QueueLayanan03 antrian = new QueueLayanan03(5);
        int pilihan;

        do {
            System.out.println("\n=== LAYANAN UNIT KEMAHASISWAAN ===");
            System.out.println("1. Tambah Antrian Mahasiswa (Mendaftar)");
            System.out.println("2. Panggil Antrian (Proses Layanan)");
            System.out.println("3. Cek Antrian Terdepan dan Terakhir");
            System.out.println("4. Cek Jumlah Mahasiswa Mengantre");
            System.out.println("5. Cek Status Antrian (Kosong/Penuh)");
            System.out.println("6. Kosongkan Semua Antrian");
```

```
            System.out.println("0. Keluar Program");
            System.out.print("Pilih menu (0-6): ");
            pilihan = sc.nextInt();
            sc.nextLine();

            switch (pilihan) {
                case 1:
                    if (!antrian.isFull()) {
                        System.out.print("Masukkan NIM  : ");
                        String nim = sc.nextLine();
                        System.out.print("Masukkan Nama : ");
                        String nama = sc.nextLine();
                        // c. Pendaftaran data mahasiswa
                        MahasiswaLayanan03 mhsBaru = new MahasiswaLayanan03(nim, nama);
                        antrian.enqueue(mhsBaru);
                    } else {
                        System.out.println("Antrian penuh! Tidak bisa menambah data.");
                    }
                    break;
                case 2:
                    antrian.dequeue();
                    break;
                case 3:
                    antrian.printFrontAndRear();
                    break;
                case 4:
                    antrian.printQueueCount();
                    break;
                case 5:
                    System.out.println("Apakah antrian kosong? " + (antrian.isEmpty() ? "Ya" :
"Tidak"));
                    System.out.println("Apakah antrian penuh?  " + (antrian.isFull() ? "Ya" :
"Tidak"));
                    break;
                case 6:
                    antrian.clear();
                    break;
                case 0:
                    System.out.println("Keluar dari program. Terima kasih!");
                    break;
                default:
                    System.out.println("Pilihan tidak valid. Silakan coba lagi.");
            }
        } while (pilihan != 0);

        sc.close();
    }
}
```